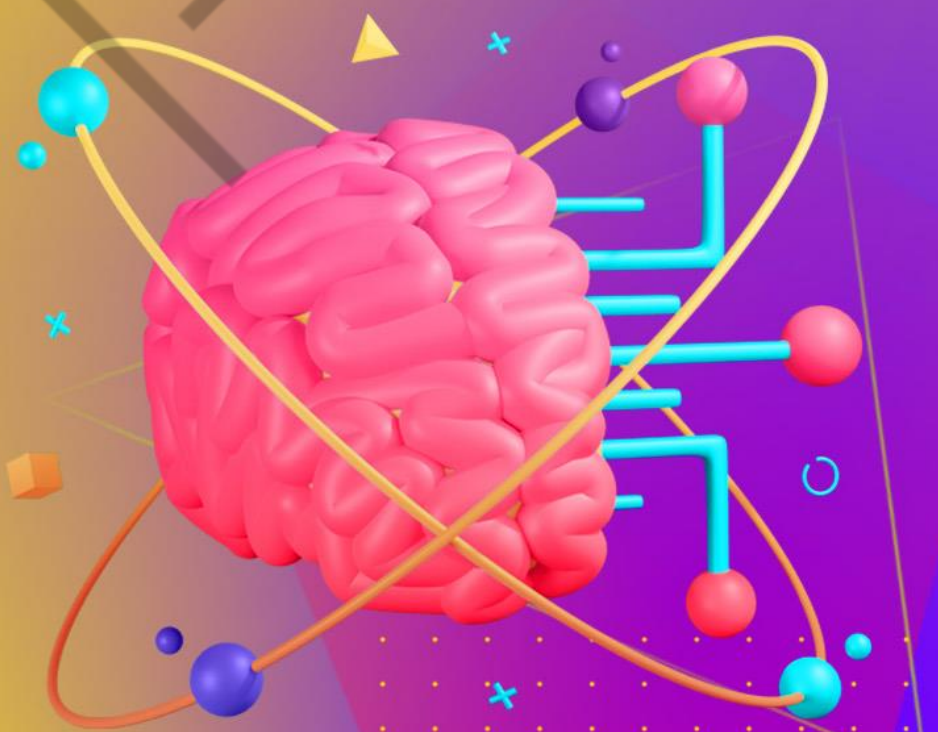
FIAP ON

BUILDING STATISTICAL MODELS

REDES NEURAIS MULTICAMADAS



6

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Tabela verdade e representação visual do “AND”.....6
Figura 2 – Tabela verdade e representação visual do “OR”.....7
Figura 3 – Tabela verdade e representação visual do “OR”.....7
Figura 4 – Representação de uma rede neural artificial multicamada.....9
Figura 5 – Funções de distribuição de probabilidade: (a) uniforme, (b) normal gaussiana.....17
Figura 6 – Imagens representativa do MNIST Dataset. 19
Figura 7 – Rede neural para classificação do MNIST Dataset.....20
Figura 8 – Exemplo de equação do segundo grau.....23
Figura 9 – Exemplos de equações complexa.....24
Figura 10 – Separação de dados em treino, validação e teste 26
Figura 11 – Até quando treinar redes neurais multicamadas 27
Figura 12 – Problemas dos gradientes.....32
Figura 13 – Resultado do código-fonte 2 35
Figura 14 – Resultado do código-fonte 4 37
Figura 15 – Resultado do código-fonte 5 38
Figura 16 – Representação da nossa primeira Rede Neural Artificial Multicamadas 39
Figura 17 – Resultado do código-fonte 6 40
Figura 18 – Resultado do código-fonte 7 41
Figura 19 – Resultado do código-fonte 8 42
Figura 20 – Resultado do código-fonte 9 44
Figura 21 – Resultado do código-fonte 10 46
Figura 22 – Resultado do código-fonte 11 47
Figura 23 - Resultado do código-fonte 12 48

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Cálculo do número de pesos e bias da rede neural da Figura 7 21

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports iniciais 34
Código-fonte 2 – Importação do dataset e montagem do DataFrame.....34
Código-fonte 3 – Divisão dos dados em treino e teste 36
Código-fonte 4 – Definição e treinamento do Perceptron.....36
Código-fonte 5 – Plotando o Iris Dataset.....37
Código-fonte 6 – Primeiro modelo com Rede Neural Multicamadas 39
Código-fonte 7 – Segundo modelo com Rede Neural Multicamadas 41
Código-fonte 8 – Terceiro modelo com Rede Neural Multicamadas 42
Código-fonte 9 – Quarto modelo com Rede Neural Multicamadas 43
Código-fonte 10 – Primeira proposta de solução para o desafio.....45
Código-fonte 11 – Segunda proposta de solução para o desafio.....46
Código-fonte 12 – Terceira proposta de solução para o desafio 47

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

REDES NEURAIS MULTICAMADAS 6
1 INTRODUÇÃO 6
1.1 Por que precisamos de redes multicamadas? 6
1.2 Componentes de redes multicamadas 8
1.3 Vocabulário de redes multicamadas 10
1.3.1 Taxa de aprendizagem 11
1.3.2 Rede Feedforward 11
1.3.3 Época 11
2 ARQUITETURAS 12
2.1 Escolha da topologia 13
2.2 Escolha da quantidade de camadas 13
2.3 Escolha da quantidade de neurônios 14
2.4 Demais escolhas 15
3 INICIALIZAÇÃO DE PESOS 16
3.1 Estratégias de inicialização 16
3.2 Inicialização de He 17
3.3 Inicialização de Xavier 18
4 TREINAMENTO DA REDE 18
4.1 Introdução e estratégias de treinamento 18
4.2 Backpropagation 22
4.3 Até quando treinar redes neurais multicamadas? 26
5 OVERFITTING E REGULARIZAÇÃO 28
6 HIPERPARÂMETROS 29
7 DESAFIOS E LIMITAÇÕES 31
8 MÃO NA MASSA! REDES MÚLTIPLAS CAMADAS EM PYTHON 33
9 HANDS ON: MODELANDO PROBLEMAS COMPLEXOS COM REDES MULTICAMADAS 44
9.1 Seu desafio 44
9.2 Proposta de solução 45
REFERÊNCIAS 50
GLOSSÁRIO 51

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REDES NEURAIS MULTICAMADAS

1 INTRODUÇÃO

1.1 Por que precisamos de redes multicamadas?

Em nossa jornada pelo mundo das redes neurais, já conhecemos o Perceptron, um componente fundamental nessa aventura. O Perceptron é excelente para resolver problemas lineares, como a separação (classificação) de pontos em um plano com uma linha reta.

Imagine que você precisa determinar se um cliente fará uma compra com base em seu histórico de compras anteriores. Se o histórico for representado por um único número, como o total de compras, você pode usar um Perceptron para criar uma linha de decisão simples que separa os clientes que compram daqueles que não compram.

No nosso exemplo, utilizamos o Perceptron para modelar o “E LÓGICO” (AND), cuja tabela verdade e representação visual estão abaixo:

A	B	A e B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

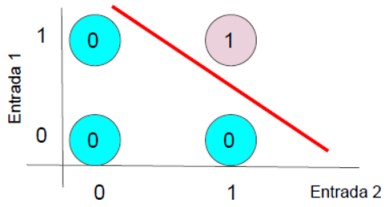


Figura 1 – Tabela verdade e representação visual do “AND”
Fonte: Imagem adaptada de https://blog.portaldomedico.com/anatomia-do-cerebro/

O lado direito da Figura 1 mostra que o “E LÓGICO” pode ser linearmente modelado como um problema de classificação representado pela reta em vermelho. Outro exemplo que se encaixa nessa definição é a representação do “OU LÓGICO” (OR), analogamente representado pela Figura 2:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A	B	A ou B
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

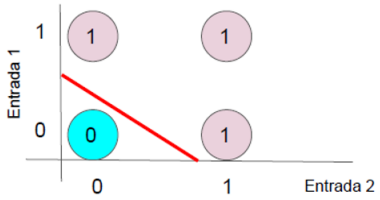


Figura 2 – Tabela verdade e representação visual do “OR”
Fonte: Imagem adaptada de https://blog.portaldomedico.com/anatomia-do-cerebro/

O lado direito da Figura 2 mostra que o “OU LÓGICO” também pode ser linearmente modelado como um problema de classificação representado pela reta em vermelho.

No entanto, **o mundo real está repleto de problemas muito mais complexos e não lineares**. Imagine agora um problema em que você precisa classificar imagens de gatos e cachorros. Cada imagem pode ser representada por pixels, e simplesmente traçar uma linha reta para separar gatos de cachorros seria uma tarefa impossível. É aqui que entram as redes neurais multicamadas.

Na nossa analogia com representações lógicas, um problema mais complexo que o “AND” ou o “OR” é o “XOR” (“OU EXCLUSIVO”), no qual a resposta somente é verdadeira se *apenas uma* das condições for verdadeira. A Figura 3 ilustra a tabela verdade e a representação visual do “XOR”:

A	B	A xor B
TRUE	TRUE	FALSE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

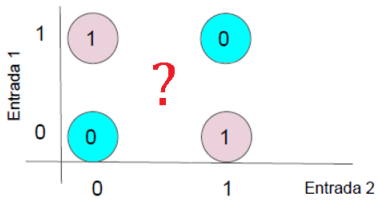


Figura 3 – Tabela verdade e representação visual do “OR”
Fonte: Imagem adaptada de https://blog.portaldomedico.com/anatomia-do-cerebro/

A representação visual do “XOR” na Figura 3 deixa claro que não há como “traçar uma reta” que separe adequadamente as regiões da classe ativada (1) da classe não ativada (0). Portanto, precisamos de representações de mais alto nível (mais complexas) para modelar esse tipo de problema.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A solução é adicionar mais camadas às redes neurais, tornando-as "multicamadas". As redes multicamadas são projetadas para lidar com problemas não lineares. Elas são compostas por várias camadas de neurônios interconectados, o que permite a criação de representações mais complexas e abstratas dos dados. No caso das imagens de gatos e cachorros, veremos adiante que as redes multicamadas podem aprender automaticamente a identificar características como bordas, texturas e padrões, tornando a tarefa de classificação muito mais precisa.

Portanto, precisamos de redes multicamadas para enfrentar os desafios do mundo real, onde os problemas são frequentemente complexos e não lineares. Elas nos permitem construir modelos mais poderosos e flexíveis, capazes de lidar com uma ampla variedade de tarefas, desde reconhecimento de fala até tradução automática, e muito mais.

1.2 Componentes de redes multicamadas

As redes neurais multicamadas são compostas por vários componentes que trabalham juntos para aprender e modelar relações complexas nos dados. Sua representação básica está ilustrada na Figura 4:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

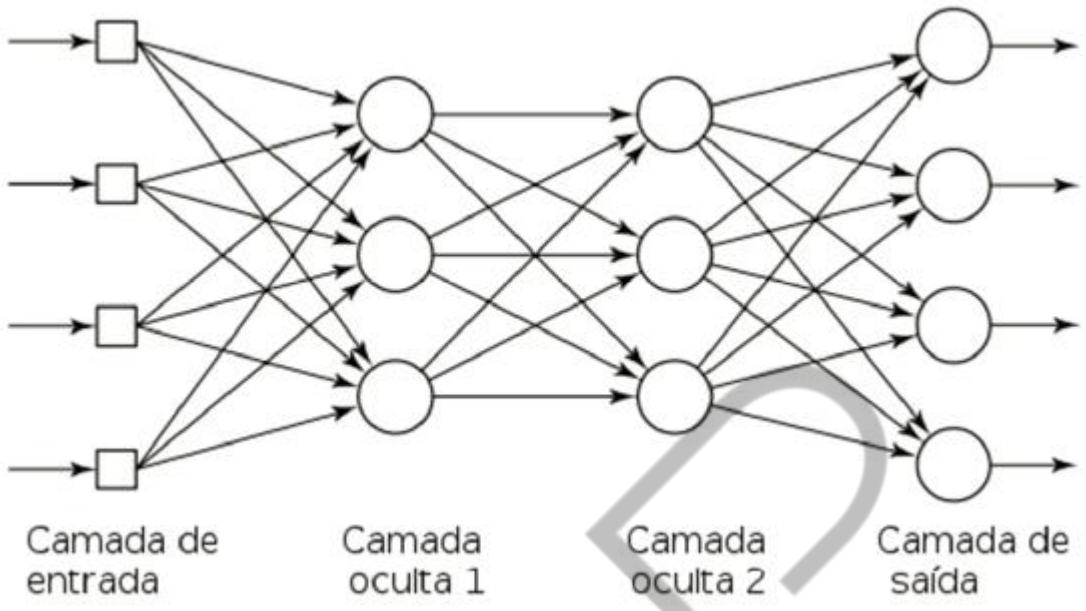


Figura 4 – Representação de uma rede neural artificial multicamada
Fonte: Imagem adaptada de https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/

Vamos explorar esses componentes essenciais. Primeiramente, temos a definição das **camadas.** Existem três tipos principais de camadas:

- **Camada de Entrada (*Input Layer*)**: essa é a primeira camada da rede, onde os dados de entrada são alimentados na rede. Cada nó nessa camada representa uma característica (*feature*) de entrada;
- **Camadas Ocultas (*Hidden Layers*)**: são as camadas intermediárias entre a camada de entrada e a camada de saída. Essas camadas não são diretamente acessíveis, e são onde ocorre a transformação dos dados. A Figura 4 mostra 2 camadas ocultas. Podemos ter mais ou menos camadas ocultas;
- **Camada de Saída (*Output Layer*)**: a última camada da rede, que produz as saídas desejadas ou predições. O número de neurônios nessa camada depende do tipo de problema (classificação, regressão etc.). No exemplo da Figura 4, temos 4 neurônios na camada de saída, o que simboliza um problema de classificação com 4 classes distintas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Notamos que os neurônios estão conectados. Cada conexão entre neurônios é associada a um peso, por isso essas conexões são denominadas "**conexões ponderadas**" (*weighted connections*). Os pesos determinam a importância das informações transmitidas de um neurônio para outro durante o processo de aprendizado.

Além do que mostra a Figura 4, não podemos deixar de mencionar as funções relevantes nas arquiteturas de redes neurais multicamadas, que são:

- **Funções de Ativação (*Activation Functions*)**: cada neurônio, exceto os de entrada, aplica uma função de ativação aos seus inputs ponderados. Essas funções introduzem não-linearidades na rede, permitindo que ela aprenda a representar relações complexas nos dados;
- **Função de Perda (*Loss Function*)**: a função de perda é usada para avaliar o quão bem a rede está performando em relação às saídas desejadas. O objetivo durante o treinamento é minimizar essa função, ajustando os pesos e *biases* (vieses).

Veremos também que para treinar a rede neural, é necessário um **algoritmo de otimização** que ajuste os pesos e biases com base na função de perda. Além disso, as redes neurais também envolvem conceitos de **regularização** (para evitar overfitting) e ajustes de **hiperparâmetros** (como taxa de aprendizado) para otimizar o treinamento. Como esses são conceitos mais avançados e extremamente importantes, veremos com detalhes ainda neste capítulo.

Esses são os principais componentes de uma rede neural multicamada. Cada um desempenha um papel crucial no processo de aprendizado e na capacidade da rede de modelar relações complexas nos dados.

1.3 Vocabulário de redes multicamadas

Quando aprendemos ou revisitamos os conceitos das redes neurais multicamadas, já vimos uma porção de termos e seus significados. Além deles, também precisamos complementar nosso vocabulário de redes multicamadas com

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

alguns termos importantes. São eles: “taxa de aprendizagem”, “redes feedforward” e “época”.

1.3.1 Taxa de aprendizagem

A taxa de aprendizagem (*learning rate*) é um hiperparâmetro crucial em redes neurais. Ela controla o tamanho dos ajustes feitos nos pesos da rede durante o treinamento. Uma taxa de aprendizagem alta significa que os pesos são ajustados em grandes passos a cada iteração, o que pode fazer com que a rede convirja rapidamente, mas também pode resultar em oscilações e problemas de estabilidade.

Por outro lado, uma taxa de aprendizagem baixa faz com que os ajustes nos pesos sejam pequenos, levando a convergência lenta e possivelmente ficando presa em mínimos locais. A escolha da taxa de aprendizagem ideal depende do problema específico e geralmente requer experimentação.

1.3.2 Rede Feedforward

Uma rede feedforward, também conhecida como rede neural artificial de alimentação direta, é o tipo mais simples e comum de rede neural. Nesse tipo de rede, **as informações fluem apenas em uma direção, da camada de entrada para a camada de saída, sem formar ciclos**.

Cada camada de neurônios está conectada à próxima camada, e cada conexão tem um peso associado. Esses pesos são ajustados durante o treinamento para que a rede aprenda a mapear os inputs para as saídas desejadas. As redes feedforward são eficazes em muitos tipos de problemas de aprendizado supervisionado, como classificação e regressão.

1.3.3 Época

Uma época (*epoch*) em treinamento de rede neural refere-se a uma única passagem por todo o conjunto de treinamento. Em outras palavras, durante uma época, todos os exemplos de treinamento são usados para atualizar os pesos da rede uma vez. O treinamento de uma rede neural geralmente envolve várias épocas, onde

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

a rede é exposta repetidamente aos mesmos exemplos de treinamento. Isso é feito para ajustar gradualmente os pesos da rede, refinando sua capacidade de fazer predições precisas.

As épocas são um conceito importante, pois o número de épocas afeta diretamente o processo de treinamento:

- Um **número insuficiente de épocas** pode resultar em subajuste (underfitting), onde a rede não aprende bem os padrões nos dados.
- Um **número excessivo de épocas** pode levar ao sobreajuste (overfitting), onde a rede se ajusta demais aos dados de treinamento e não generaliza bem para novos dados.

Portanto, encontrar o número ideal de épocas é uma parte crítica do treinamento de redes neurais.

2 ARQUITETURAS

Acabamos de conhecer os componentes típicos de uma rede neural multicamadas. Agora **aprenderemos a configurar a arquitetura das redes neurais** multicamadas. A arquitetura da rede é definida por um conjunto de escolhas que são feitas antes do início do treinamento. Uma vez que a arquitetura da rede está montada, ela não é alterada em tempo de treinamento.

Para exemplificar a diferença, note que o treinamento da rede é descobrir o valor adequado de cada peso. Portanto, o valor do peso se altera em tempo de treinamento. Mas, se definirmos que a arquitetura da rede em questão terá 4 camadas ocultas, tal característica permanecerá imutável ao longo do treinamento.

Definir a arquitetura da rede neural refere-se ao processo de especificar sua estrutura, ou seja, o **número de camadas**, o **número de neurônios** em cada camada e a **forma como essas camadas estão interconectadas (topologia)**. A arquitetura é um dos principais fatores que determinam a capacidade e a capacidade de generalização de uma rede neural.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.1 Escolha da topologia

A topologia se refere à organização geral das camadas. A mais comum é rede **completamente conectada** (*fully connected*), representada na Figura 4. Nessa topologia, cada neurônio ou unidade em uma camada está conectado a todos os neurônios na camada seguinte. Isso significa que todas as saídas de uma camada são usadas como entradas para cada neurônio na camada seguinte, criando uma rede densa de conexões.

A escolha da topologia correta depende do problema que você está tentando resolver e dos dados disponíveis. É uma parte fundamental do projeto de uma rede neural e geralmente envolve experimentação e ajustes para encontrar a configuração que melhor se adapta ao seu problema específico.

Além das redes completamente conectadas, existem arquiteturas como redes convolucionais (CNNs) para processamento de imagens e redes recorrentes (RNNs) para sequências temporais. Por enquanto, focaremos nas redes feedforward completamente conectadas.

2.2 Escolha da quantidade de camadas

As redes neurais geralmente consistem em uma camada de entrada (*input*), uma ou mais camadas ocultas (*hidden layers*) e uma camada de saída (*output*).

A escolha do número de camadas ocultas em uma rede neural é um aspecto crítico do projeto e treinamento da rede, pois afeta diretamente sua capacidade de modelar relacionamentos complexos nos dados.

De forma geral, considere os seguintes aspectos:

- **Natureza do problema**: a escolha do número de camadas ocultas deve ser orientada pelo problema que você está tentando resolver. Problemas simples podem ser tratados com uma única camada oculta, enquanto problemas mais complexos podem exigir múltiplas camadas;
- **Complexidade dos dados**: se os dados contiverem padrões não lineares complexos, camadas ocultas adicionais podem ajudar a

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

capturar esses padrões. Se os dados forem relativamente simples e lineares, camadas adicionais podem ser desnecessárias;

- **Experimentação**: a experimentação desempenha um papel fundamental na determinação do número de camadas ocultas. Você pode começar com uma arquitetura simples e, se a rede não estiver aprendendo bem ou estiver subajustando (underfitting) os dados, considerar adicionar camadas ocultas para aumentar a capacidade da rede;
- **Empirismo**: o aprendizado de máquina muitas vezes envolve uma dose de empirismo. Experimente diferentes arquiteturas, desde redes com uma única camada oculta até redes com várias camadas ocultas, e avalie o desempenho em um conjunto de validação para determinar qual arquitetura funciona melhor para o seu problema.

Em resumo, a escolha do número de camadas ocultas é uma tarefa que envolve experimentação e ajustes. Não há uma regra rígida que se aplique a todos os cenários, e a seleção ideal depende da natureza do problema e dos dados. Portanto, é importante estar disposto a iterar e otimizar a arquitetura da rede com base no desempenho observado.

2.3 Escolha da quantidade de neurônios

A escolha da quantidade de neurônios em cada camada também é um fator debatível. Não há uma regra 100% assertiva nessa escolha, pois ela também depende do problema sendo modelado e dos dados empregados no treinamento da rede.

Jeff Heaton, no seu livro "*Introduction to Neural Networks for Java*", sugere três abordagens para escolha da quantidade de neurônios em uma camada oculta. Para efeitos de exemplificação, considere uma rede com 30 neurônios na camada de entrada e 2 neurônios na camada de saída. As abordagens são:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1. **O número de neurônios ocultos deve estar entre o tamanho da camada de entrada e o da camada de saída**. No nosso exemplo, o valor de (30+2)/2 = 16 neurônios;
2. **O número de neurônios escondidos deve ser 2/3 do tamanho da camada de entrada, mais o tamanho da camada de saída**. Assim, a camada escondida no nosso exemplo deve conter (30*2/3)+2 = 22 neurônios;
3. **O número de neurônios escondidos deve ser menor que o dobro do tamanho da camada de entrada**. Ou seja, no nosso exemplo, a camada escondida deve conter menos que 60 neurônios

Qualquer uma dessas abordagens é um bom "chute" para criar sua arquitetura inicial. *E se tivermos mais de uma camada oculta?* Para mais de uma camada oculta, basta seguir o raciocínio usando a primeira camada oculta como entrada para a próxima.

Lembre-se de que a escolha da quantidade de neurônios não é uma regra fixa e pode variar de problema para problema. É importante iterar e experimentar diferentes arquiteturas para encontrar aquela que melhor funciona para o seu caso.

2.4 Demais escolhas

Além de definir previamente a topologia, número de camadas e tamanho das camadas ocultas, existem outros aspectos que precisam ser decididos antes de iniciarmos o treinamento das redes neurais multicamadas:

- **Funções de ativação**: escolher as funções de ativação para cada camada é uma parte importante da arquitetura. As funções de ativação introduzem não-linearidades na rede, permitindo que ela modele relações não lineares nos dados;
- **Política de atualização da taxa de aprendizado**: até agora, mencionamos apenas a taxa de aprendizagem como um valor fixo a ser

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

utilizado como medida de “velocidade” do aprendizado dos pesos e bias. No entanto, existem políticas de atualização da taxa de aprendizado. Cada política altera a taxa de aprendizado de uma forma, que pode contribuir (ou não) para melhor ajuste dos pesos. Veremos essas configurações nas configurações de hiperparâmetros.

3 INICIALIZAÇÃO DE PESOS

Uma parte crucial no treinamento de redes neurais profundas é a inicialização eficiente dos pesos. Nesse processo, valores iniciais são atribuídos aos pesos das conexões entre neurônios, influenciando diretamente o desempenho da rede.

A forma como os pesos são inicializados afeta a convergência e estabilidade do treinamento. Pesos muito pequenos podem dificultar a aprendizagem, enquanto pesos muito grandes podem levar a mínimos locais. Por outro lado, inicializar todos os pesos com zero ou valores muito pequenos pode levar a problemas de convergência, já que todas as unidades aprenderão quase a mesma coisa. Experimentação é essencial para encontrar a melhor configuração.

3.1 Estratégias de inicialização

As principais estratégias de inicialização são:

- **Inicialização Aleatória**: pesos são definidos com valores aleatórios em um intervalo específico, sendo simples, mas inconsistente;
- **Inicialização com Distribuição Uniforme**: valores são amostrados de uma distribuição uniforme em um intervalo, sendo simples, mas inconsistente;
- **Inicialização com Distribuição Normal**: os pesos são amostrados de uma distribuição normal com média zero e desvio padrão específico, proporcionando estabilidade.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

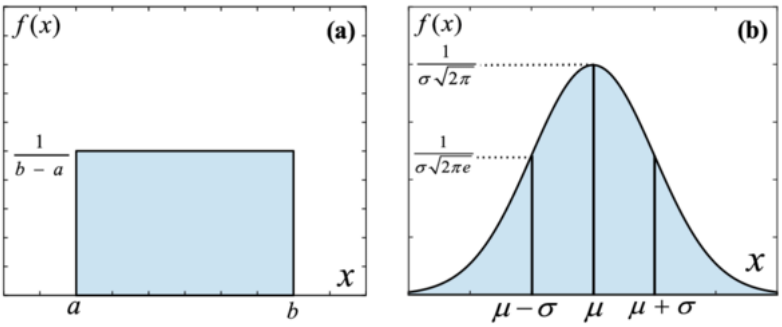


Figura 5 – Funções de distribuição de probabilidade: (a) uniforme, (b) normal gaussiana
Fonte: Imagem reproduzida de https://www.researchgate.net/figure/Probability-distribution-function-a-uniform-distribution-of-the-excitation-frequency_fig5_339429775

A Figura 5 ilustra a diferença entre distribuições uniformes e normais. Enquanto a distribuição uniforme pode levar a pesos similares, a distribuição normal ameniza esse problema.

- **Heurísticas Específicas**: alguns tipos de redes neurais, como as convolucionais (que aprenderemos adiante), utilizam heurísticas específicas, como a de He ou Xavier, levando em conta entradas e saídas.

3.2 Inicialização de He

Desenvolvida para unidades de ativação ReLU (Rectified Linear Unit), a Inicialização de He define os pesos iniciais usando uma distribuição normal com média zero e variância 2/n, onde "*n*" é o número de entradas para a unidade.

Isso ajuda a manter a ativação inicial em uma faixa útil, evitando o problema de gradientes que desaparecem ou explodem (tipos de problemas que aprenderemos logo mais).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3.3 Inicialização de Xavier

Essa técnica é adequada para funções de ativação sigmoide ou tangente hiperbólica. Os pesos são inicializados com uma distribuição normal com média zero e variância 2/(n_in + n_out), onde "*n_in*" é o número de unidades de entrada e "*n_out*" é o número de unidades de saída. Isso ajuda a manter a variância constante por meio das camadas da rede.

4 TREINAMENTO DA REDE

4.1 Introdução e estratégias de treinamento

Vimos que problemas complexos exigem redes mais complexas. Para nos guiar nos conceitos de treinamento de redes neurais multicamadas, imagine que tenhamos imagens de dígitos escritos à mão e queremos reconhecer automaticamente que dígito é esse.

O problema de reconhecer dígitos escritos à mão é uma tarefa de classificação de padrões em visão computacional e aprendizado de máquina. Consiste em desenvolver um modelo capaz de identificar e classificar corretamente dígitos manuscritos, geralmente de 0 a 9, em imagens digitais. Cada imagem contém um único dígito manuscrito que pode variar em estilo, tamanho, inclinação e posicionamento dentro da imagem. A Figura 6 ilustra parte do MNIST, um dataset popular construído para treinar algoritmos nessa tarefa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 6 – Imagens representativa do MNIST Dataset.
Fonte: Imagem reproduzida de https://en.wikipedia.org/wiki/MNIST_database

O objetivo é criar um sistema que possa "ler" e identificar os dígitos escritos à mão com alta precisão, independentemente da caligrafia do autor. Esse problema é relevante em várias aplicações do mundo real, como sistemas de reconhecimento de caracteres em documentos digitalizados, classificação automática de códigos postais em correspondências, reconhecimento de números de telefone em formulários manuscritos, entre outros.

Claramente não podemos tratar o problema com o Percpetron, exigindo uma rede neural mais sofisticada. Vamos utilizar a rede *parcialmente* representada na Figura 7, indicando uma rede feedforward, totalmente conectada, contendo 4 camadas: a entrada com 784 neurônios, a 1ª camada oculta com 16 neurônios, a 2ª camada oculta com 32 neurônios e a camada de decisão com 10 neurônios. Não podemos representar a rede completa, pois a imagem seria um caos!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

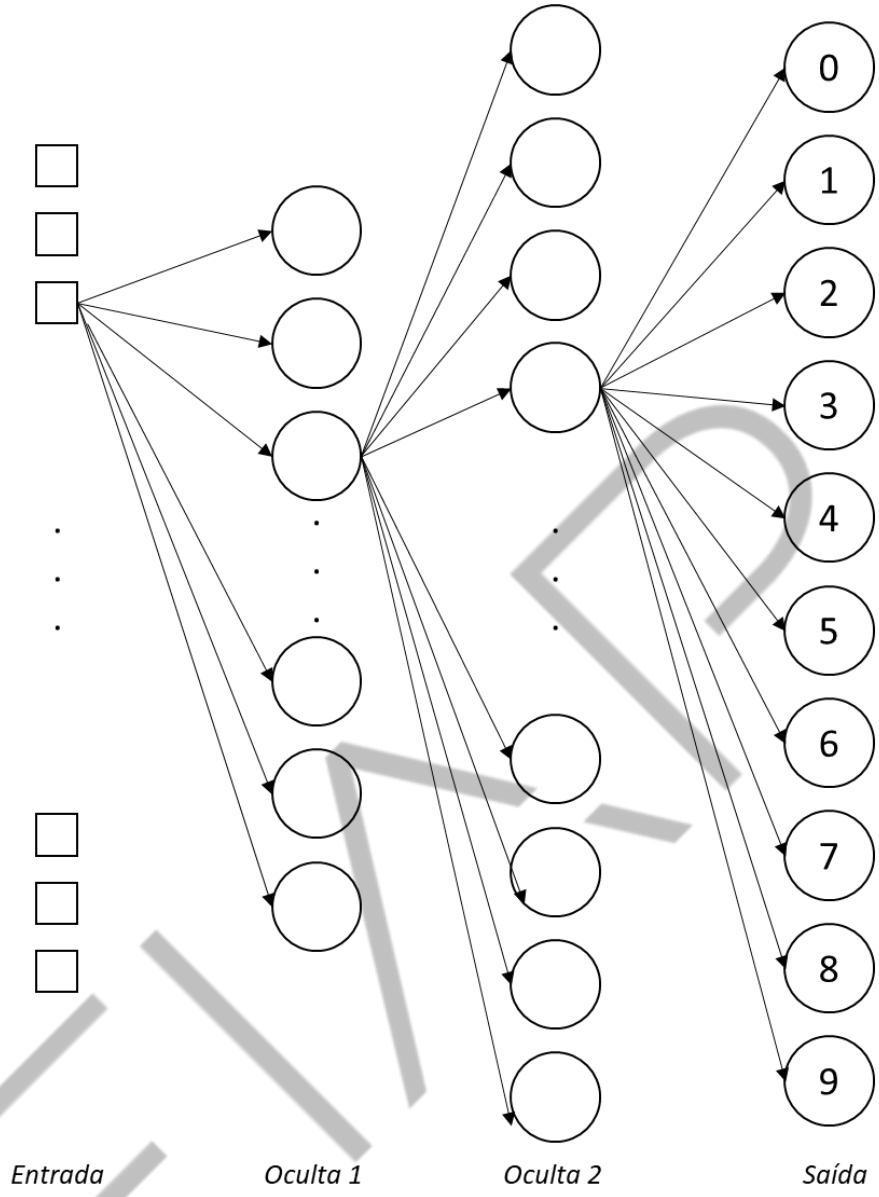


Figura 7 – Rede neural para classificação do MNIST Dataset.
Fonte: Autor (2023)

<u>Por que essa representação?</u> As imagens do MNIST possuem 28x28 pixels, assim, cada imagem contém 784 pixels. A camada de entrada contém um neurônio para cada pixel da entrada. A camada de saída obrigatoriamente precisa ter 10 neurônios, pois temos um problema de classificação com 10 classes (cada um dos dígitos entre 0 e 9). Escolhemos 2 camadas ocultas de forma arbitrária, e a quantidade de neurônios em cada uma delas seguiu a heurística proposta por Jeff Heaton.

<u>Quantos pesos e bias são necessários na arquitetura proposta?</u> Tente calcular! A solução está no quadro abaixo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Cálculo do número de pesos e bias da rede ilustrada na Figura 7

Para calcular o número total de pesos e bias necessários para essa rede neural, precisamos levar em consideração as conexões entre as camadas:
• Camada de Entrada → 1ª Camada Oculta:
  ○ Pesos: 784 (entradas) x 16 (neurônios) = 12.544 pesos
  ○ Bias: 16 (neurônios) = 16 bias
• 1ª Camada Oculta → 2ª Camada Oculta:
  ○ Pesos: 16 (neurônios da camada anterior) x 32 (neurônios da camada atual) = 512 pesos
  ○ Bias: 32 (neurônios da camada atual) = 32 bias
• 2ª Camada Oculta → Camada de Saída:
  ○ Pesos: 32 (neurônios da camada anterior) x 10 (neurônios de saída) = 320 pesos
  ○ Bias: 10 (neurônios de saída) = 10 bias
Portanto, o número total de pesos é 12.544 (da primeira camada) + 512 (da segunda camada) + 320 (da camada de saída) = **13.376 pesos**, e o número total de bias é 16 (da primeira camada) + 32 (da segunda camada) + 10 (da camada de saída) = **58 bias**.

Quadro 1 – Cálculo do número de pesos e bias da rede neural da Figura 7
Fonte: Autor (2023)

<u>Como encontrar os pesos e bias do problema apresentado?</u> Podemos usar a *força bruta*, tentando combinações diferentes para valores dos pesos e bias, até encontrar aquela que mais minimizaria o erro de classificação. Certamente essa estratégia levaria muito, mas muito tempo, para convergir e não é nada prática.

Vimos que podemos atualizar os pesos de acordo com uma fórmula que depende dos valores correntes, mas se o peso inicial está “ruim” e tentarmos “corrigir” na direção oposta à esperada, os novos valores estarão cada vez mais distantes do que deveriam ser. Portanto, precisamos de uma estratégia mais robusta!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A estratégia robusta a ser empregada é denominada ***backpropagation*** (retropropagação). O Backpropagation é um algoritmo fundamental no treinamento de redes neurais artificiais. Ele é usado para ajustar os pesos das conexões entre neurônios de forma a minimizar o erro entre as saídas previstas pela rede e os valores reais desejados. O processo envolve a retropropagação do erro da camada de saída para as camadas ocultas, ajustando gradualmente os pesos em direção a um mínimo global na função de erro.

Comparando com a força bruta, o Backpropagation é um método muito mais eficiente e inteligente para treinar redes neurais. Enquanto a força bruta envolveria tentativas exaustivas de ajustar os pesos de forma aleatória e verificar os resultados, o Backpropagation utiliza informações precisas sobre o erro da rede em relação aos dados de treinamento. Isso permite que a rede aprenda de maneira mais rápida e eficaz, evitando ajustes aleatórios e demorados.

4.2 Backpropagation

O backpropagation, ou retropropagação, é o motor por trás do treinamento de redes neurais artificiais multicamadas. Sua intuição é ajustar os pesos das conexões entre neurônios de forma a minimizar o erro entre as saídas previstas pela rede e os valores reais desejados. A pergunta que nos direciona é:

Como atualizar os pesos da rede de forma eficiente (rápida),
sem prejudicar as métricas de desempenho?

Antes de mais nada, devemos lembrar das principais funções relacionadas com as redes neurais:

- **Função de ativação**: retorna se um neurônio será ativado ou não;
- **Função de perda**: calcula o erro daquela predição (resposta correta – resposta calculada);
- **Função de custo**: erro médio de aprendizagem (média dos resultados da função de perda).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Repare que a “função de custo” pode ser reescrita utilizando apenas pesos, bias e funções de ativação, pois ela é a média dos erros, que depende do valor calculado baseado na função de ativação (a qual depende da soma ponderada das *entradas*, *pesos* e do *bias*).

Portanto, “treinar a rede neural” passa a ser encarado como um problema de otimização: **como podemos otimizar a função de custo?** Felizmente, existem meios matemáticos para isso!

Vamos começar com o conceito de “**vetor gradiente**”. O gradiente é uma ferramenta matemática que indica a direção na qual obtém-se o **crescimento máximo** de uma função qualquer. Observe a Figura 8: considerando que y = f(x), como podemos variar “x” de modo a incrementar “y”?

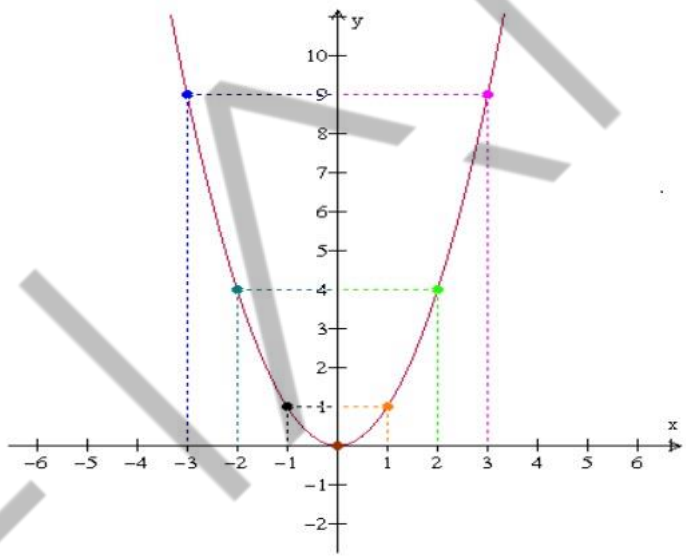


Figura 8 – Exemplo de equação do segundo grau
Fonte: Imagem reproduzida de https://deverdecasa-web.blogspot.com/2017/05/grafico-da-funcao-do-2-grau.html

De acordo com a função ilustrada na Figura 8, para maximizar essa função (aumentar “y”), podemos *aumentar o valor de “x”* (pois f(x=3) > f(x=2)) ou até mesmo *diminuir o valor de “x”* (pois f(x=-3) > f(x=-2)).

Esse caso foi fácil, basta olhar para a função que o “sentido” da maximização está posto. No entanto, em funções mais complexas, encontrar tal “sentido” pode não ser tão óbvio, como ilustrado na Figura 9:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

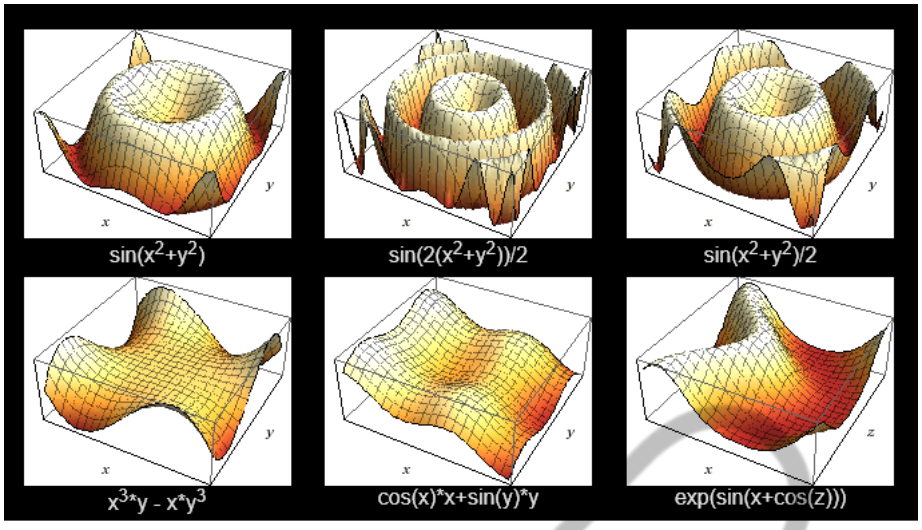


Figura 9 – Exemplos de equações complexa
Fonte: Imagem adaptada de https://www.fractarte.com.br/wolframalpha/3d-equations.php

Nesses casos, como podemos obter o vetor gradiente? Por meio de **derivadas**! A derivada de uma função é uma medida de quão rápido o valor da função muda quando a variável independente muda. Em termos práticos, diz-nos a inclinação da função em qualquer ponto, indicando se a função está aumentando, diminuindo ou permanecendo constante naquele ponto.

Mas o nosso objetivo é **reduzir** o erro. Para minimizar a função de erro, queremos seguir na direção oposta ao gradiente. Isso é conhecido como **gradiente descendente**.

Imagine uma montanha com uma bola rolando nela. O vetor gradiente descendente é como um guia que diz à bola para rolar para baixo, em direção ao ponto mais baixo da montanha, que representa o mínimo global da função de erro.

Até o momento, imagino que você tenha entendido os artifícios matemáticos que otimizam uma função, e os usamos para otimizar a função de custo. **Mas como empregamos, de fato, o vetor gradiente descendente calculado para atualizar os pesos e bias?**

Para responder a essa questão, considere uma rede simples com 2 pesos (p1, p2) e 1 bias (b). Imagine que numa determinada iteração do treinamento, o vetor gradiente descendente foi [0,5; 50; -2.3], respectivamente, para [p1; p2; b]. É importante destacar que o vetor gradiente descendente não revela o quanto atualizar cada parâmetro, mas sua direção e intensidade. Da seguinte forma:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Peso 1 (0,5)**: o gradiente descendente é positivo, o que indica que aumentar o peso 1 diminuiria o custo;
- **Peso 2 (50)**: o gradiente é muito maior em magnitude comparado ao peso 1, o que sugere que o peso 2 tem um impacto muito maior no custo. Novamente, porque o gradiente descendente é positivo, queremos aumentar esse peso para diminuir o custo;
- **Bias (-2,3):** o gradiente descendente é negativo, o que implica que diminuir o bias reduziria o custo. Portanto, para minimizar o custo, devemos diminuir o valor corrente do bias.

Na prática, faríamos essas atualizações multiplicando cada componente do gradiente descendente pela taxa de aprendizado e então somando o resultado dos valores atuais dos parâmetros. Se **α** é a taxa de aprendizado, a regra de atualização para cada parâmetro **θ** seria:

$$\theta_{atualizado} = \theta_{corrente} + \alpha * (\text{gradiente descendente correspondente})$$

Agora precisamos relembrar um ponto crucial: *queremos minimizar o erro médio*, logo, devemos passar por todos os exemplos do conjunto de treino. Mas, e se o conjunto de treino for muito grande? É daí que surgiu o conceito de **vetor gradiente descendente estocástico**.

No cálculo do vetor gradiente descendente estocástico não usamos todos os dados do treino de uma vez para atualizar os pesos. Nós separamos aleatoriamente o treino em subconjuntos denominados **batches** (lotes). Calculamos o vetor gradiente descente médio apenas para um batch. Na próxima atualização usamos outro batch, e assim sucessivamente. Após utilizarmos todos os batches disponíveis, dizemos que completamos uma época do treinamento. Daí voltamos a usar o primeiro batch, depois, o segundo e assim por diante, por quantas épocas forem necessárias para convergir o treinamento.

Para finalizar, vale ressaltar um detalhe: é claro que o processo de otimização da função de custos por meio do backpropagation é um processo complexo e rico em

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

detalhes matemáticos. Aqui passamos a intuição de funcionamento dessa técnica. Adiante, veremos como tais estratégias de otimização são escolhidas na montagem da arquitetura da rede e seus impactos nos resultados.

4.3 Até quando treinar redes neurais multicamadas?

Treinar redes neurais multicamadas envolve encontrar um equilíbrio delicado entre subtreinar (underfitting) e supertreinar (overfitting) o modelo. Em outras palavras, queremos que a rede aprenda o suficiente para fazer previsões precisas, mas não tanto a ponto de memorizar os dados de treinamento. O ponto ideal de parada do treinamento pode variar dependendo do problema, do tamanho do conjunto de dados e da arquitetura da rede, mas existem algumas estratégias gerais para determinar quando parar.

Uma abordagem comum é o uso de dados de validação. Durante o treinamento, uma parte dos dados é separada para validação (Figura 9). Podemos fazer essa divisão explicitamente ou delegar para que as próprias bibliotecas de Machine Learning façam essa divisão antes de iniciar o treinamento.

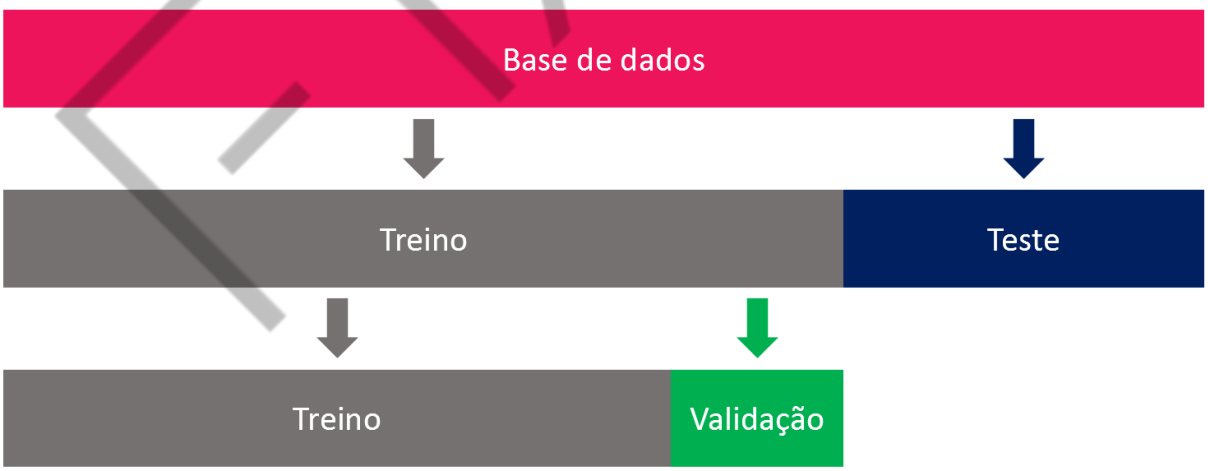


Figura 10 – Separação de dados em treino, validação e teste
Fonte: autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

À medida que o treinamento avança, o desempenho do modelo na validação é monitorado. Quando o desempenho na validação para de melhorar ou começa a piorar, isso pode ser um sinal de que o modelo está começando a supertreinar (overfitting). Nesse ponto, interrompe-se o treinamento para evitar que a rede memorize os dados de treinamento e não generalize bem para novos dados (Figura 10).

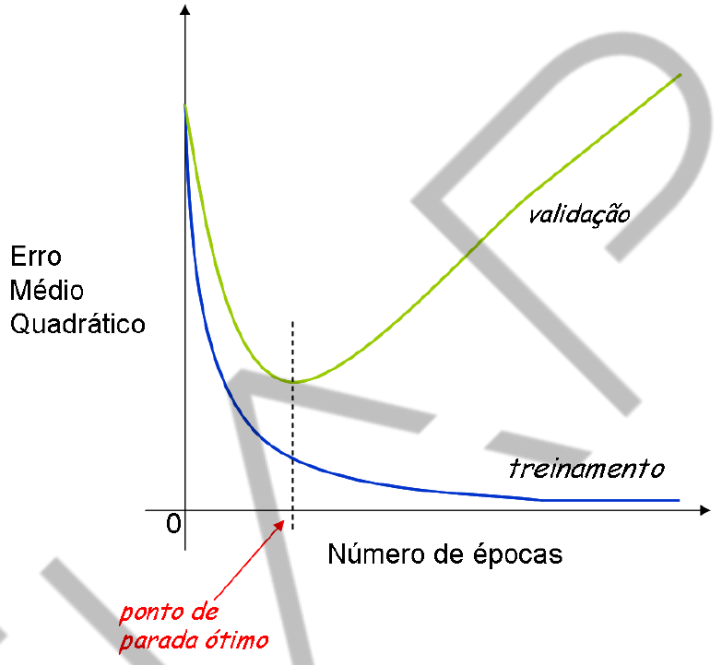


Figura 11 – Até quando treinar redes neurais multicamadas
Fonte: autor (2024)

A Figura 11 ilustra o ponto ótimo de parada do treinamento da rede neural: durante o treinamento, o erro associado ao conjunto de treinamento tende sempre a diminuir, assim como o erro associado ao conjunto de validação (que está atuando como um teste). A partir de um determinado ponto, a rede continuaria aprendendo diminuindo o erro do treinamento, mas passaria a aumentar o erro da validação, indicando a memorização do treino (overfitting). A parada no momento logo anterior a esse ponto é a parada ideal.

Outra técnica é o **uso de regularização**, como a regularização L1 ou L2, que adiciona uma penalidade aos pesos da rede para evitar que se tornem muito grandes. Isso ajuda a controlar o overfitting e permite que o treinamento seja estendido por mais épocas sem comprometer o desempenho.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Além disso, é importante estar ciente dos recursos computacionais disponíveis. Treinar redes neurais multicamadas pode ser intensivo em termos de tempo e recursos. Portanto, em alguns casos, pode ser necessário interromper o treinamento devido a restrições de tempo ou hardware. Isso é implementado **indicando o número máximo de épocas que a rede pode treinar**.

Concluindo, o momento ideal para interromper o treinamento de redes neurais multicamadas é quando o desempenho na validação não melhora mais ou começa a piorar, indicando overfitting. Empregar estratégias para evitar overfitting (como acompanhar as métricas de validação e/ou usar técnicas de regularização) é essencial para determinar o momento de parada do treinamento.

5 OVERFITTING E REGULARIZAÇÃO

O overfitting é o fenômeno no qual o modelo se ajusta muito bem aos dados de treinamento, mas não consegue generalizar bem para novos dados. Isso ocorre quando o modelo memoriza os dados de treinamento em vez de capturar padrões mais amplos que se aplicam a novos dados.

A regularização é uma técnica utilizada para evitar o overfitting, **introduzindo uma penalidade nos pesos do modelo durante o treinamento**. Isso ajuda a controlar a complexidade do modelo, impedindo que os pesos se tornem muito grandes.

Existem duas técnicas de regularização comuns, L1 e L2:

- Na **regularização L1**, a penalidade é proporcional à *soma dos valores absolutos dos pesos*. Isso tem o efeito de "encorajar" alguns pesos a se tornarem zero, o que pode resultar em modelos mais esparsos;
- Na **regularização L2**, a penalidade é proporcional à *soma dos quadrados dos pesos*. Isso tende a "encorajar" os pesos a serem menores, mas não exatamente zero.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Um pouco de estatística: o termo "L" em L1 e L2 se refere à norma utilizada para calcular a penalidade. Na regularização L1, a norma utilizada é a norma L1, que é a soma dos valores absolutos dos pesos. Na regularização L2, a norma utilizada é a norma L2, que é a raiz quadrada da soma dos quadrados dos pesos.

A escolha entre regularização L1 e L2 depende do problema específico e das características dos dados. A regularização L1 tende a produzir modelos mais esparsos, o que pode ser útil se houver muitas features irrelevantes ou redundantes. Por outro lado, a regularização L2 tende a produzir modelos mais suaves, o que pode ser preferível em problemas nos quais a complexidade do modelo não é uma preocupação.

Em resumo, a regularização é uma técnica importante para evitar o overfitting em modelos de aprendizado de máquina, e as técnicas de regularização L1 e L2 são ferramentas valiosas para controlar a complexidade do modelo. A escolha entre as duas técnicas depende das características do problema e dos dados.

6 HIPERPARÂMETROS

Ao montarmos a arquitetura de uma rede neural multicamada, há uma série de hiperparâmetros a serem considerados. Muitos deles podem ter seus valores estimados por meio de heurísticas (como a escolha da quantidade de camadas ocultas e de neurônios por camada), mas, no fundo, todos são dependentes dos dados em questão.

Portanto, a **otimização de hiperparâmetros** em redes neurais também é tão relevante quanto de qualquer outro método de aprendizado de máquina. O maior desafio aqui está na quantidade de hiperparâmetros sendo ajustados, bem como no tempo de treinamento de cada combinação.

Para facilitar a jornada de entendimento de hiperparâmetros, vamos listar os principais a serem considerados, indicando o que significam, valores tradicionalmente

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

explorados e potenciais impactos nos resultados. Tomaremos como referência os nomes de hiperparâmetros adotados pela biblioteca scikit-learn. São eles:

- **`hidden_layer_sizes`**: especifica a arquitetura da rede neural, ou seja, o número de camadas ocultas e o número de neurônios em cada camada oculta. Por exemplo, `hidden_layer_sizes=(100, 50)` indica uma rede neural com duas camadas ocultas, a primeira com 100 neurônios e a segunda com 50 neurônios;
- **`activation`**: define a função de ativação a ser usada em cada camada oculta. As opções comuns incluem "relu" para a ativação ReLU (Rectified Linear Unit), "logistic" para a função logística (sigmóide) e "tanh" para a tangente hiperbólica;
- **`solver`**: especifica o algoritmo de otimização a ser usado para treinar a rede neural. Alguns dos solvers disponíveis incluem "adam" para o otimizador Adam, "sgd" para o Gradiente Descendente Estocástico e "lbfgs" para o método Quasi-Newton com Limited-memory Broyden-Fletcher-Goldfarb-Shanno (mais indicado para bases pequenas);
- **`alpha`**: controla a força da regularização L2 aplicada durante o treinamento. Quanto maior o valor de alpha, mais forte é a regularização;
- **`learning_rate`**: especifica a política de atualização da taxa de aprendizado usada pelo algoritmo de otimização;
- **`learning_rate_init`**: taxa de aprendizado inicial usada pelo algoritmo de otimização. Esse hiperparâmetro é usado apenas se learning_rate for definido como "constant";
- **`max_iter`**: define o número máximo de iterações (épocas) durante o treinamento da rede neural. Se o algoritmo de otimização não convergir antes de atingir esse número máximo de iterações, o treinamento será interrompido;
- **`batch_size`**: tamanho do lote usado durante o treinamento da rede neural. Esse hiperparâmetro especifica o número de amostras que serão usadas em cada iteração do algoritmo de otimização;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **`shuffle`**: indica se deve embaralhar as amostras em cada iteração do treinamento;
- **`random_state`**: controla a semente usada pelo gerador de números aleatórios;

- **`tol`**: tolerância para a convergência do algoritmo de otimização (valor da diferença de performance entre uma iteração e outra);
- **`verbose`**: indica se deve imprimir mensagens de progresso durante o treinamento;
- **`early_stopping`**: indica se deve parar o treinamento quando o desempenho não melhorar em uma iteração pelo conjunto de validação.

Esses são apenas alguns dos hiperparâmetros disponíveis no scikit-learn para controlar o treinamento de redes neurais multicamadas. Encorajamos a consulta das documentações oficiais para a lista completa e maior aprofundamento de sua utilização.

Vale lembrar que ajustar tais hiperparâmetros pode ter um grande impacto no desempenho e na capacidade de generalização da rede neural. Experimentar diferentes combinações de valores para esses hiperparâmetros e avaliar o desempenho do modelo usando técnicas de validação cruzada é uma parte importante do processo de ajuste de hiperparâmetros.

7 DESAFIOS E LIMITAÇÕES

As redes neurais, apesar de sua versatilidade e capacidade de lidar com uma ampla gama de problemas, também enfrentam diversos desafios e limitações que podem impactar seu desempenho e eficácia em determinadas situações.

Os principais desafios enfrentados pelas redes neurais estão listados abaixo e representados na Figura 12:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- "**vanishing gradient" (desaparecimento do gradiente)**: ocorre quando os gradientes das funções de ativação se tornam muito pequenos à medida que são propagados para camadas anteriores da rede durante o treinamento. Isso pode resultar em uma atualização insignificante dos pesos nessas camadas, dificultando a aprendizagem de padrões complexos e profundidades maiores;
- **"exploding gradient" (explosão do gradiente)**: ocorre quando os gradientes se tornam muito grandes, levando a atualizações de pesos excessivamente grandes e instabilidade no treinamento da rede.

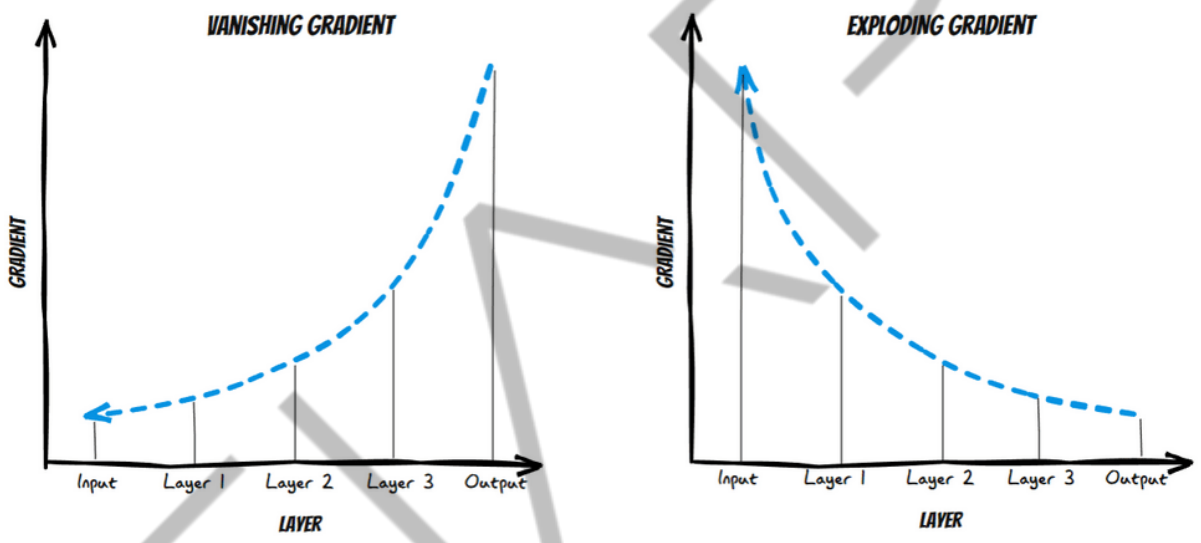


Figura 12 – Problemas dos gradientes
Fonte: Imagem reproduzida de https://medium.com/dscier/how-to-deal-with-vanishing-and-exploding-gradients-in-neural-networks-24eb00c80e84

As redes neurais também têm suas limitações. Embora sejam capazes de aprender representações complexas de dados e realizar tarefas sofisticadas, elas podem não ser a escolha ideal para todos os problemas. Em casos em que os dados são escassos ou mal rotulados, outras abordagens de aprendizado de máquina, como árvores de decisão, métodos baseados em regras ou modelos lineares, podem ser mais adequadas.

Além disso, as redes neurais podem ser computacionalmente intensivas e exigir grandes conjuntos de dados para treinamento, o que pode tornar seu uso impraticável em certos cenários. Além disso, o processo de ajuste de hiperparâmetros

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

e a seleção da arquitetura adequada da rede podem ser desafiadores e requerem experiência e experimentação cuidadosas.

Portanto, embora as redes neurais sejam uma ferramenta poderosa e versátil em aprendizado de máquina, é importante reconhecer seus desafios e limitações e considerar cuidadosamente se são a melhor escolha para um determinado problema. Em muitos casos, uma abordagem mais simples e interpretação de modelo pode ser mais eficaz e apropriada.

8 MÃO NA MASSA! REDES MÚLTIPLAS CAMADAS EM PYTHON

Agora que conhecemos as Redes Neurais Multicamadas, está na hora de utilizarmos na prática para modelarmos problemas de classificação. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: veja o material HOW TO para acessar o ambiente do Colab.

DICA: uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 4) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

DICA: Para essa parte do código, utilizaremos um dataset já salvo na memória do scikit-learn, por isso não há um arquivo CSV associado!

Utilizaremos a base “Iris” já persistida no scikit-learn. Essa base é um conjunto de dados muito conhecido na área de aprendizado de máquina e classificação. Ele contém informações sobre três espécies diferentes de flores íris: setosa, versicolor e virginica. Cada amostra da base de dados inclui quatro atributos (características): comprimento e largura da sépala e comprimento e largura da pétala, todos medidos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

em centímetros. O objetivo típico ao trabalhar com o conjunto de dados Iris é criar um modelo de aprendizado de máquina que possa classificar corretamente a espécie de uma flor íris com base em suas características.

Vamos começar com os *imports*:

```
# Apenas para evitar warnings
import warnings
warnings.filterwarnings('ignore')

# Bibliotecas de Data Science
import pandas as pd
import seaborn as sns

# Separação dos dados
from sklearn.model_selection import train_test_split

# Modelos a serem utilizados
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier

# Métricas de avaliação
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score
```

Código-fonte 1 – Imports iniciais
Fonte: Elaborado pelo autor (2023)

A célula acima não produz saídas. Agora vamos puxar o dataset da memória do sklearn e montar um DataFrame Pandas com os dados. Cada variável terá uma coluna com seu nome e a última coluna, chamada “class”, será o label de cada exemplo:

```
# Lendo o montando o DataFrame com o dataset
from sklearn.datasets import load_iris
iris = load_iris()

# Transformando em Dataframe Pandas:
#   - os dados são iris.data
#   - as colunas são os nomes das características
data = pd.DataFrame( iris.data, columns=iris.feature_names )
data["class"] = iris.target
data
```

Código-fonte 2 – Importação do dataset e montagem do DataFrame
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Ao executar o código acima você verá o DataFrame ilustrado na Figura 13, com 150 linhas (exemplos) e 5 colunas (4 variáveis + 1 label).

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

Figura 13 – Resultado do código-fonte 2
Fonte: Elaborado pelo autor (2023)

Novamente, **como se trata de um dataset fortemente empregado para ensino de machine learning, ele já se encontra muito bem formatado: não há valores faltantes, dados duplicados ou qualquer outro "problema grave" que precisa ser sanado**. Todas as variáveis são expressas na mesma medida (centímetros) e seus valores são próximos, dispensando padronização ou normalização. Portanto, podemos pular as etapas de limpeza e exploração. Reforçamos que tais etapas são essenciais ao trabalharmos com datasets reais!

Podemos seguir diretamente na divisão dos dados em treino e teste, e aqui separaremos 20% dos dados para teste. Usaremos o "`random_state=42`" para efeitos de reprodutibilidade:

```
# Divisão em TREINO e TESTE
X = data.drop(columns=["class"]) # variáveis
y = data["class"]                # label

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
)
```

Código-fonte 3 – Divisão dos dados em treino e teste
Fonte: Elaborado pelo autor (2023)

Novamente, o código acima não produz saídas. Então, avancemos para a definição e treinamento do Perceptron, bem como a predição das saídas:

```
# Cria um perceptron e o treina (com o fit())
p = Perceptron( random_state=42 )
p.fit(X_train, y_train)


# Salva as predições do TESTE na variável y_pred
y_pred = p.predict(X_test)


# Matriz de confusão do perceptron
from sklearn.metrics import confusion_matrix

def matriz_confusao(teste_labels, teste_preds, labels):
  cm = confusion_matrix(teste_labels, teste_preds, labels=labels)
  disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                display_labels=labels)
  disp.plot(cmap="Blues")


matriz_confusao(y_test, y_pred, p.classes_)
```

Código-fonte 4 – Definição e treinamento do Perceptron
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

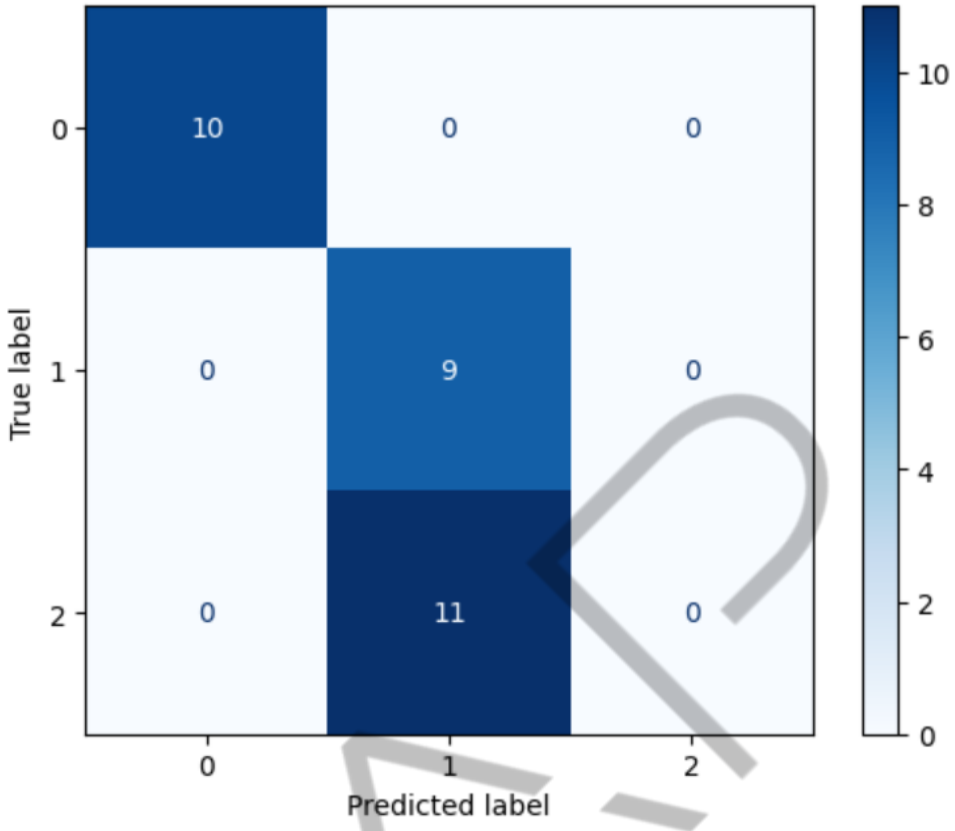


Figura 14 – Resultado do código-fonte 4
Fonte: Elaborado pelo autor (2023)

O resultado não foi muito bom, não é? Vimos que o Perceptron resolve problemas linearmente separáveis, mas será que é o caso desse dataset? Vamos investigar diversas plotagens dos dados originais:

```
sns.pairplot( data=data, vars=('sepal length (cm)', \
                               'sepal width (cm)', \
                               'petal length (cm)', \
                               'petal width (cm)'), \
              hue='class' );
```

Código-fonte 5 – Plotando o Iris Dataset
Fonte: Elaborado pelo autor (2023)

O código acima produz uma visualização muito rica que nos permite investigar o comportamento do dataset (Figura 15).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

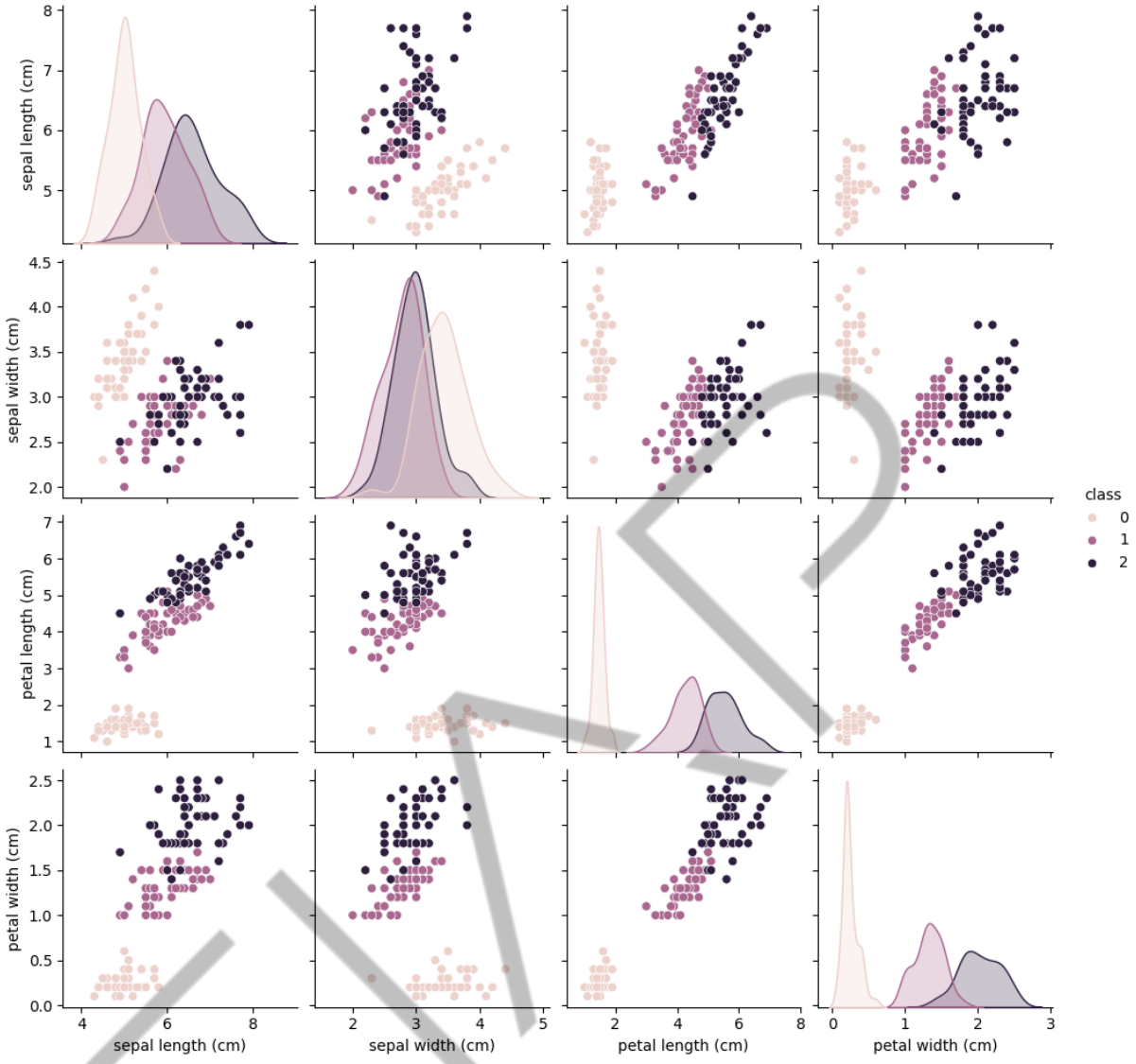


Figura 15 – Resultado do código-fonte 5
Fonte: Elaborado pelo autor (2023)

A Figura 15 mostra, sob diversos aspectos, que existe uma clara separação dos pontos da classe “0” em relação às demais, porém, as classes “1” e “2” podem ser confundidas. É exatamente esse comportamento que a matriz de confusão da Figura 13 apontou: as classes “1” e “2” são confundidas como uma única classe. Portanto, precisamos de um modelo mais sofisticado.

Nosso novo modelo será uma Rede Neural Multicamadas representada pela Figura 16, contendo apenas uma camada oculta com 5 neurônios.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

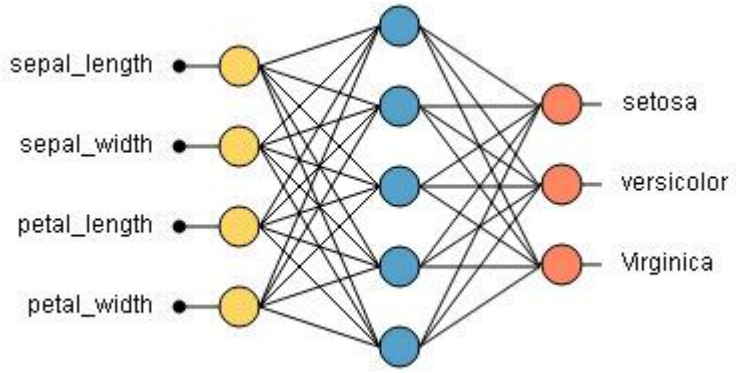


Figura 16 – Representação da nossa primeira Rede Neural Artificial Multicamadas
Fonte: Imagem reproduzida de https://miro.medium.com/v2/resize:fit:381/1*Pff-9bCGTqE6iMW2sBSTmw.png

O código para sua definição, treinamento e avaliação está abaixo. Ele define a arquitetura como esperado (1 camada oculta com 5 neurônios), mostrará as saídas na tela e treinará por no máximo 200 épocas.

```
#Arquitetura
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    verbose=True,
    max_iter=200,
    random_state=1
)

# Treinamento
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

# Avaliação
matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 6 – Primeiro modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2023)

A saída está parcialmente mostrada na Figura 17. Sua saída deve conter 200 linhas de informação (devido ao hiperparâmetro “`verbose=True`”) e terminar com a matriz de confusão abaixo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

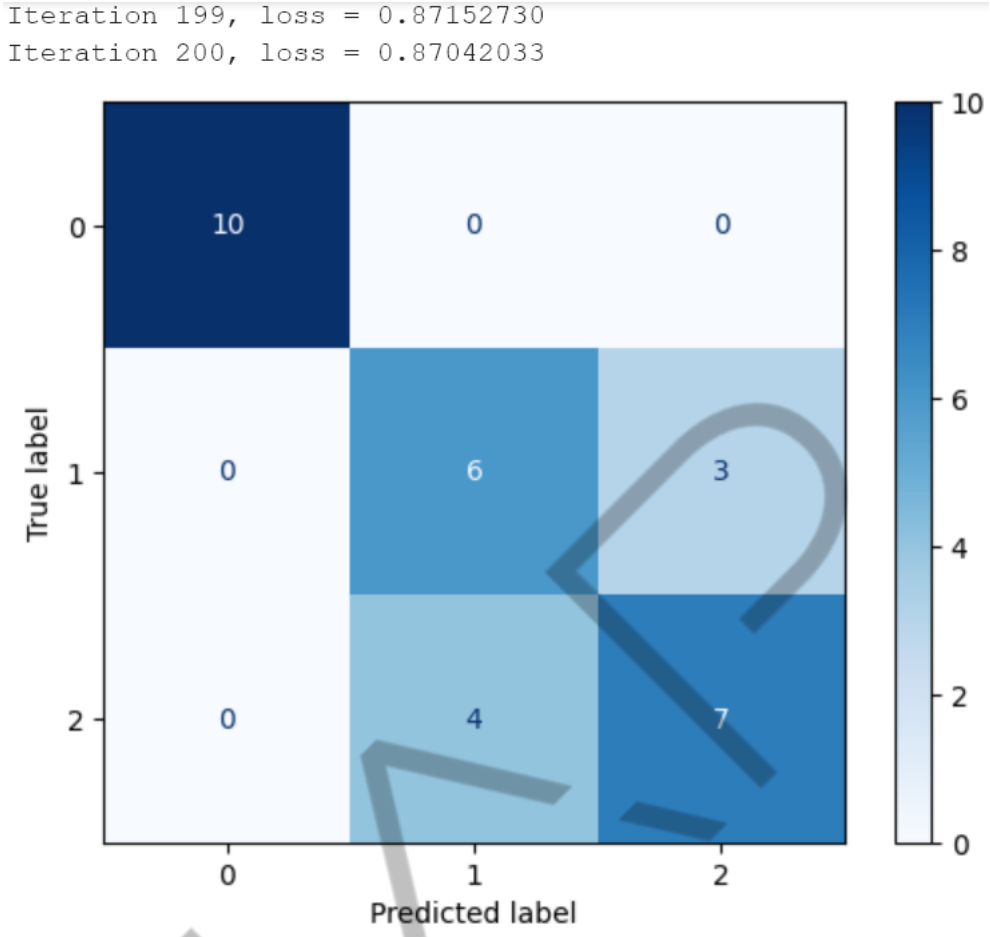


Figura 17 – Resultado do código-fonte 6
Fonte: Elaborado pelo autor (2023)

Note que o modelo proposto não fez milagre... por mais que os exemplos das classes “1” e “2” estejam começando a serem mais bem acomodados, ainda há uma mistura dos labels. Portanto, devemos tentar melhorar nosso modelo alterando os hiperparâmetros da rede neural.

Vamos começar alterando a **função de ativação**.

```
#Alterando a função de ativação...
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    verbose=False,
    max_iter=50,
    random_state=1,
    activation="logistic"
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 7 – Segundo modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2023)

E vejamos o resultado:

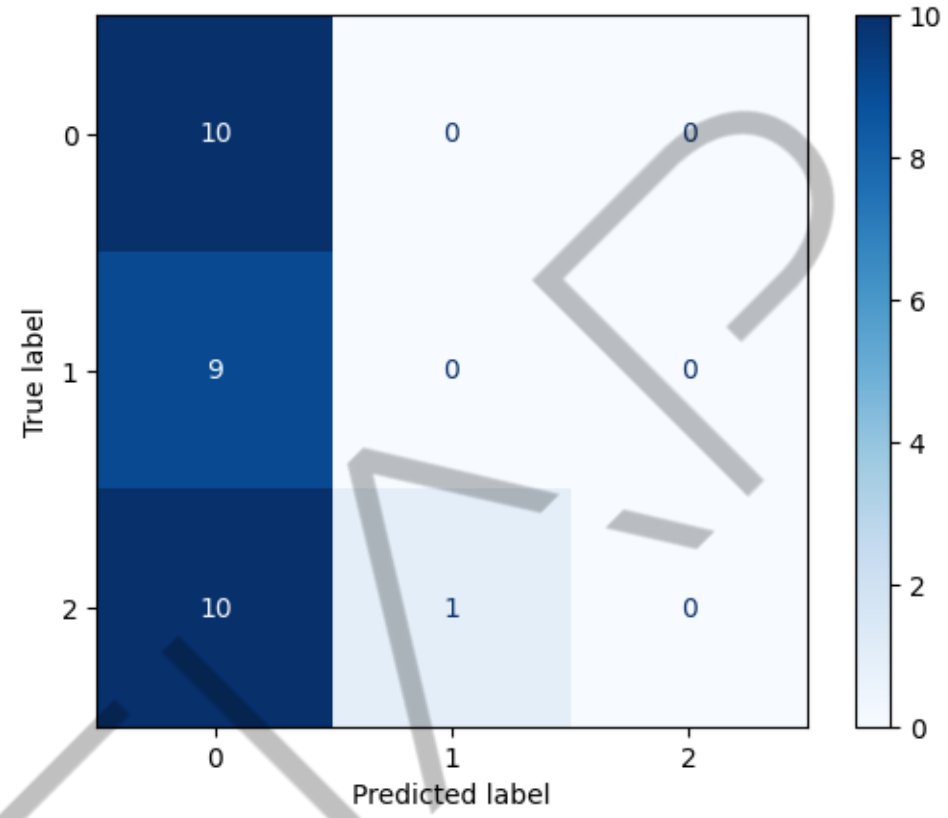


Figura 18 – Resultado do código-fonte 7
Fonte: Elaborado pelo autor (2023)

Piorou! E se também tentarmos alterar o “solver” para o algoritmo de otimização da rede mais recomendado para datasets pequenos? Essa é a dica que a própria documentação do scikit-learn nos mostra. Faremos isso com o comando:

```
#Alterando o solver para o que a própria documentação aconselha para
datasets menores
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    verbose=False,
    max_iter=50,
    random_state=1,
    activation="logistic",
    solver="lbfgs"
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 8 – Terceiro modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2023)

A saída está na Figura 19. Bem melhor! Aqui obtivemos 100% de acerto, e isso não é overfitting, pois tal métrica incrível está calculada no “teste”, ou seja, em outra base diferente do “treino”. Realmente, a rede aprendeu e soube generalizar seu conhecimento em novos dados.

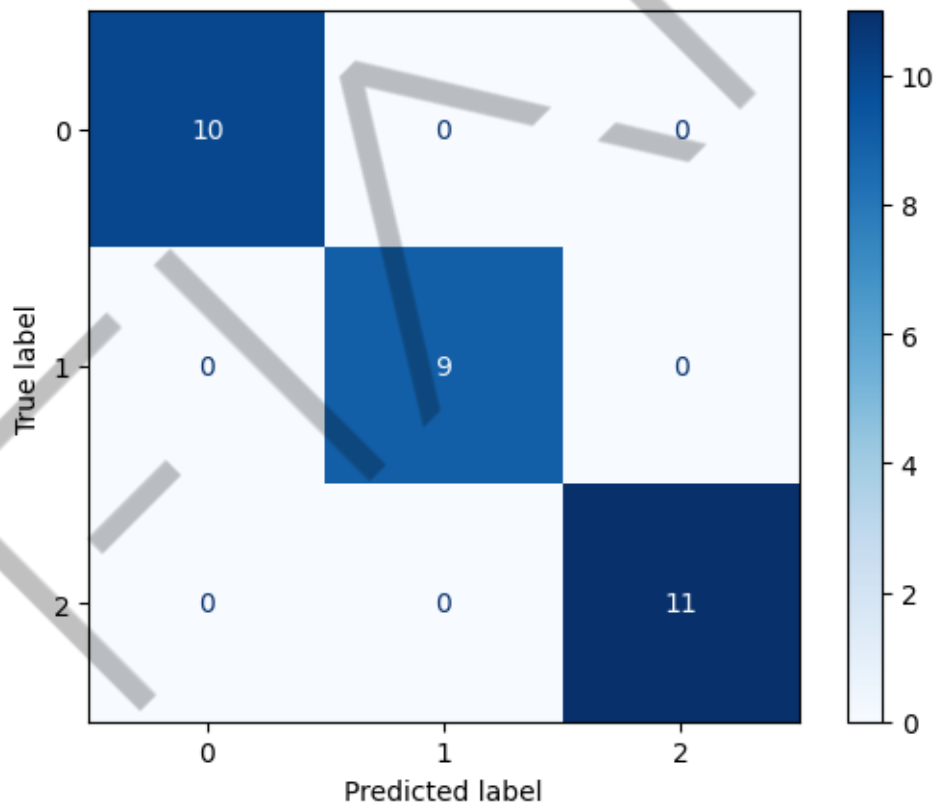


Figura 19 – Resultado do código-fonte 8
Fonte: Elaborado pelo autor (2023)

Agora, alterar o “solver” é a única saída? Não! Vimos diversos hiperparâmetros disponíveis, e certamente a combinação de outros também levaria a bons resultados. Por outro lado, outras combinações podem não ser tão boas. Vejamos o efeito de

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

combinar outro solver, com uma arquitetura mais profunda (2 camadas ocultas), e outra política de atualização da taxa de aprendizagem:

```
#Testando outra arquitetura...
mlp = MLPClassifier(
    hidden_layer_sizes=(5,3),
    max_iter=1000,
    random_state=1,
    activation="logistic",
    learning_rate="adaptive",
    solver="adam"
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 9 – Quarto modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2023)

O resultado está na Figura 20. Novamente, o resultado não foi muito bom.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

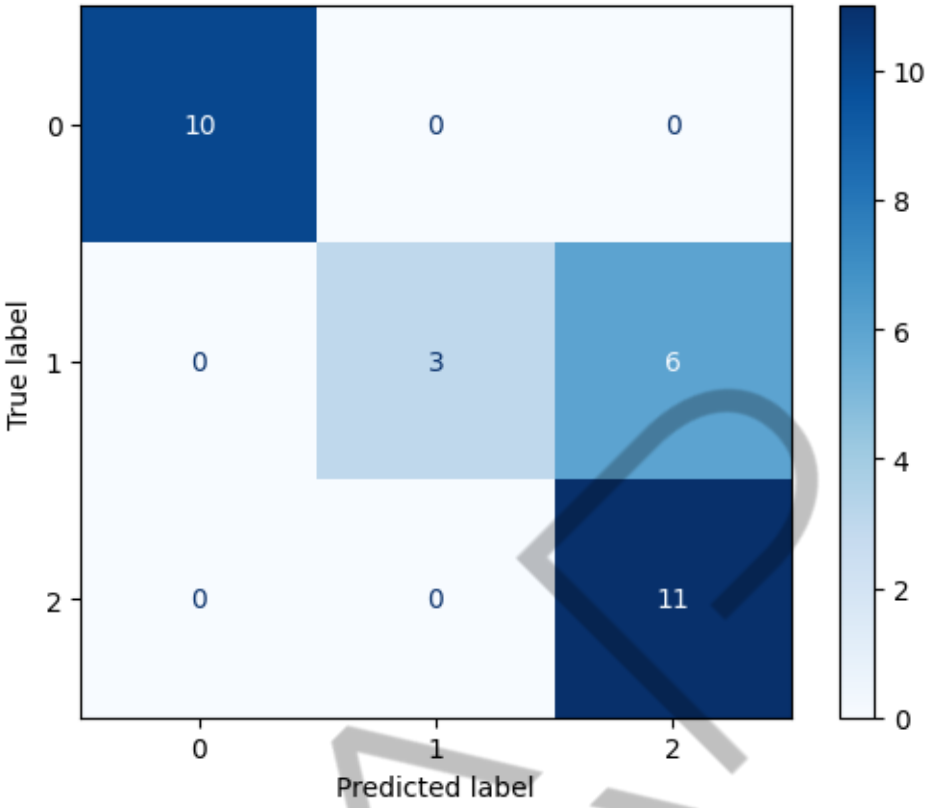


Figura 20 – Resultado do código-fonte 9
Fonte: Elaborado pelo autor (2023)

O que o último modelo nos mostra é que a quantidade de combinações de hiperparâmetros é muito grande, e que algumas delas levarão a bons resultados enquanto muitas outras não. Isso reforça a importância de realizar otimização de hiperparâmetros em validação cruzada também nos modelos de redes neurais multicamadas.

9 HANDS ON: MODELANDO PROBLEMAS COMPLEXOS COM REDES MULTICAMADAS

9.1 Seu desafio

Agora é a sua vez de criar modelos de classificação com Redes Neurais Multicamadas. Utilizando como base os códigos vistos no material, e o Iris Dataset,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

sua tarefa será investigar outras combinações de hiperparâmetros que levem a bons resultados.

9.2 Proposta de solução

Abaixo, temos três propostas de solução para o desafio apresentado. Essas são apenas algumas formas de resolver o problema. Seus modelos podem ser diferentes e com melhores/piores resultados: o importante é praticar e entender o impacto das alterações propostas nas arquiteturas.

Os códigos assumem que o notebook possui os imports, o carregamento dos dados e a definição da função "`matriz_confusao()`", como fizemos anteriormente. Comentaremos todas de uma vez no final no texto.

```
# 1ª proposta: 2 camadas ocultas, solver ADAM, função de ativação
logística
mlp = MLPClassifier(
    hidden_layer_sizes=(10, 5),
    solver="adam",
    activation="logistic",
    random_state=1
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 10 – Primeira proposta de solução para o desafio
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

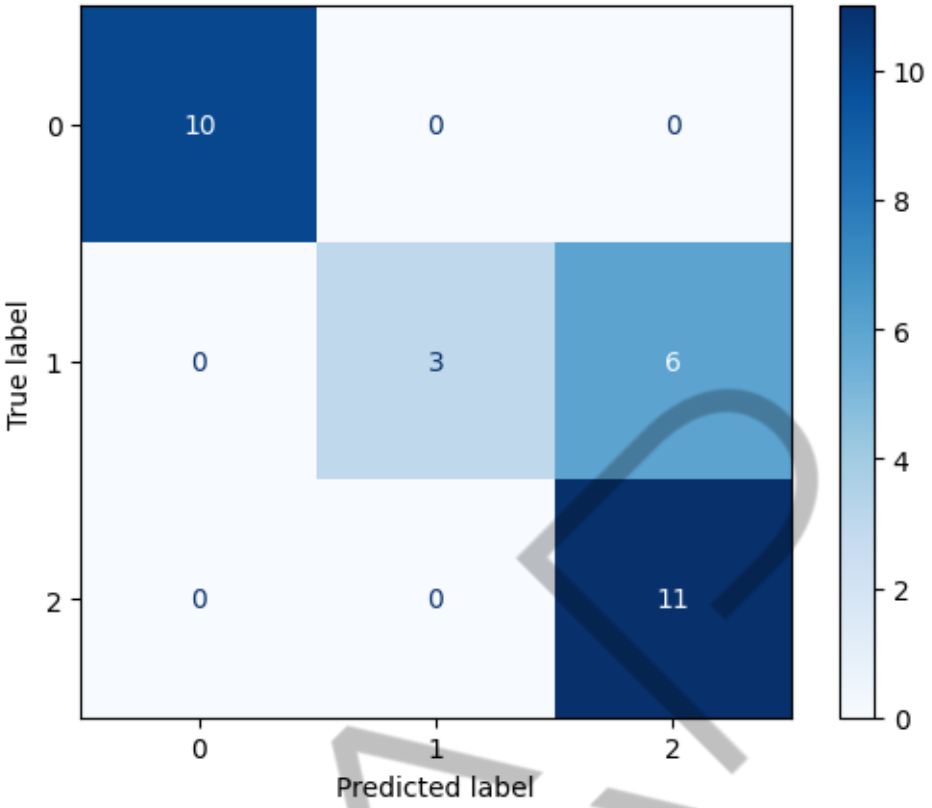


Figura 21 – Resultado do código-fonte 10
Fonte: Elaborado pelo autor (2023)

```
# 2ª proposta: 3 camadas ocultas, otimizador SGD, regularização L2 e
função de ativação relu
mlp = MLPClassifier(
    hidden_layer_sizes=(9, 7, 5),
    solver="sgd",
    activation="relu",
    alpha=0.01,
    random_state=1
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 11 – Segunda proposta de solução para o desafio
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

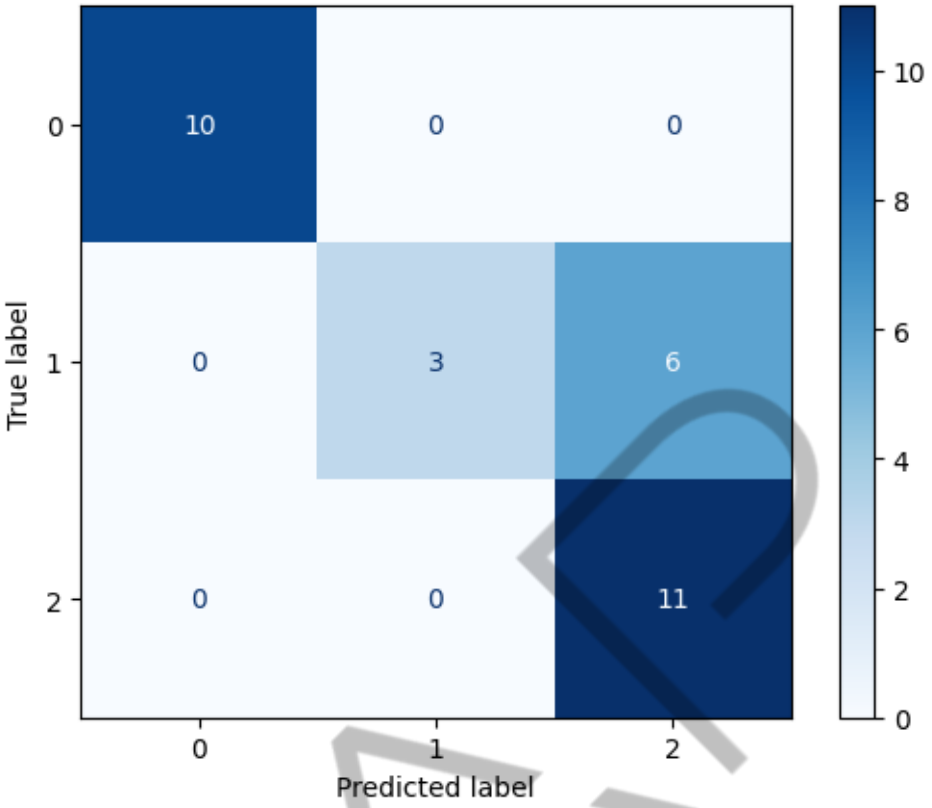


Figura 22 – Resultado do código-fonte 11
Fonte: Elaborado pelo autor (2023)

```
# 3ª proposta: 2 camadas ocultas, solver ADAM, função de ativação TANH
e regularização L2
mlp = MLPClassifier(
    hidden_layer_sizes=(10, 5),
    solver="adam",
    activation="tanh",
    alpha=0.01,
    random_state=1
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 12 – Terceira proposta de solução para o desafio
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

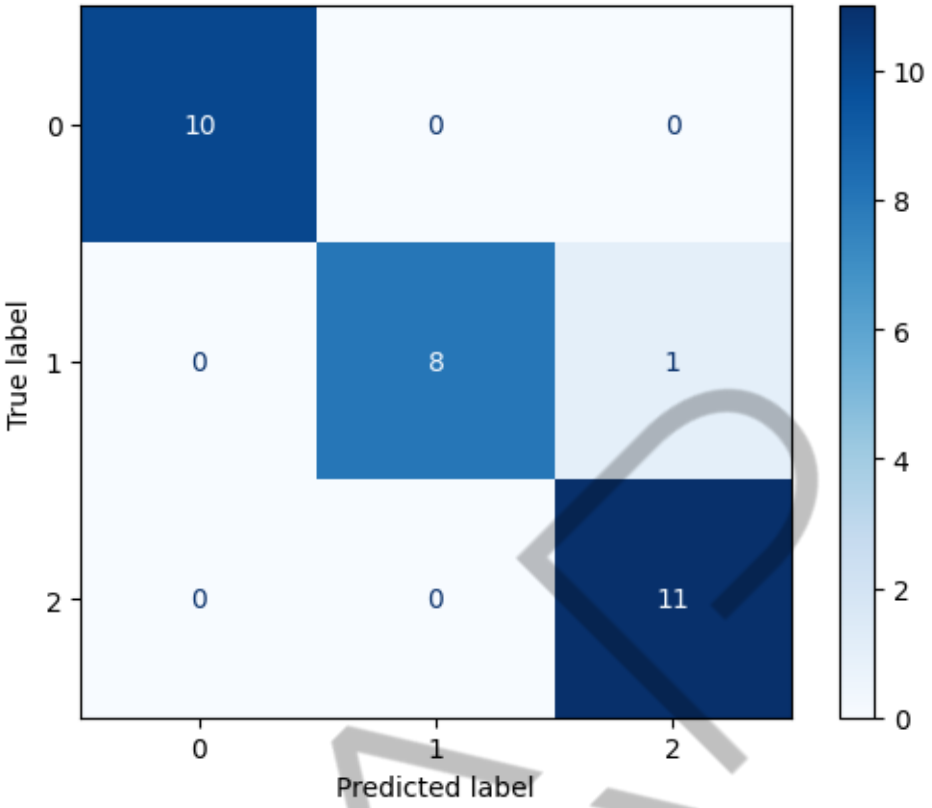


Figura 23 - Resultado do código-fonte 12
Fonte: Elaborado pelo autor (2023)

A Figura 18 já é o melhor resultado que podemos obter, afinal, não temos como melhorar acima de 100% de acurácia. No entanto, as propostas de solução acima ilustram fatos interessantes:

- A 1ª e a 2ª proposta são bem diferentes em termos de arquiteturas e levam aos mesmos resultados. Isso é comum em redes neurais;
- Mesmo sendo uma base pequena e o otimizador por vetor gradiente descendente não seja o mais indicado, a 2ª proposta prova que é possível contornar tal limitação com a exploração de outros hiperparâmetros;
- Na 3ª solução, o uso de um otimizador mais adequado e regularização levou a resultados compatíveis com os melhores obtidos até então.

A conclusão é que cada hiperparâmetro possui sua importância, e compreender seu conceito ajuda a propor arquiteturas de redes neurais multicamadas mais

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

propensas a bons resultados. Bons resultados práticos, no entanto, são dependentes das bases empregadas (dos dados!) e requerem experimentação.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

HAYKIN, Simon**. Redes neurais:** princípios e prática. Bookman Editora, 2008.

HEATON, Jeff. **Introduction to neural networks with Java**. Heaton Research, Inc., 2008.

SILVA, Ivan Nunes da; SPATTI, Danilo Hernane; FLAUZINO, Rogério Andrade. **Redes neurais artificiais para engenharia e ciências aplicadas**. 2010.

VON ZUBEN, Fernando J.; DE CASTRO, Leandro N. **Redes neurais**. Notas de aula, FEEC, Universidade Estadual de Campinas, 2003.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	Explicação.
CNN	*Convolutional Neural Network* – Rede Neural Convolucional.
ReLU	*Rectified Linear Unit* – Unidade Linear Retificada.
RNN	*Recurrent Neural Network* – Rede Neural Recorrente.
Tanh	Tangente Hiperbólica.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com